The Journal of Open Source Software

dea-tools: Geospatial analysis of satellite data using Xarray, the Open Data Cube, and Digital Earth Australia

Chad Burton [1¶], **Robbi Bishop-Taylor** [1], **Bex Dunn**[1], **Claire Phillips** [1], **Caitlin Adams**[1], **Gianluca Scortechini**[1], **Sean Chua**[1], **Vanessa Newey** [1], **Margaret Harrison**[1], **Jenna Guffogg**[1], **Claire Krause**[1], and **James Miller**[1]

1 Geoscience Australia, Australia ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

`dea-tools` is an open-source Python library that provides a comprehensive set of tools for analysing, visualising, and modelling geospatial and Earth observation data represented as xarray objects. Originally developed to support Digital Earth Australia (DEA) workflows, `dea-tools` now functions as a general-purpose library that can be used with any geospatial dataset. The package offers many utilities for spatial and temporal analysis, remote sensing index calculation, machine learning, and interactive visualisation, while integrating seamlessly with the Python geospatial stack (e.g. xarray, geopandas, and dask). Distributed as a pip-installable environment, `dea-tools` includes JupyterLab and related dependencies, providing a fully managed platform for scientific geospatial analysis on any machine. Embedded within the broader `dea-notebooks` repository, the package is supported by extensive documentation and example Jupyter notebooks demonstrating use of `dea-tools` functionality. Together, `dea-tools` and `dea-notebooks` enable reproducible, scalable, and accessible Earth observation science with Python.

Statement of need

Modern Earth observation workflows often involve complex, multi-temporal, and multi-resolution datasets that demand integrated tools for efficient loading, pre-processing, visualisation, and analysis. While foundational libraries such as `xarray`, `geopandas`, and `dask` provide the building blocks, implementing complex analysis workflows still requires substantial coding effort to handle the data processing pipeline. The `dea-tools` library addresses this gap by providing a curated suite of well-documented functions that seamlessly integrate with these core libraries.

The `dea-tools` Python package provides a comprehensive, open-source toolkit for the analysis, and interpretation of geospatial and Earth observation (EO) data represented primarily as xarray objects. Developed by Digital Earth Australia (DEA) to streamline analysis of national-scale satellite datasets, `dea-tools` has since evolved into a flexible and general-purpose library that supports many geospatial workflows. It can now be used on any system and with most gridded and vector based datasets, whether accessed from cloud-based catalogues via a Spatio-Temporal Asset Catalog (STAC), or stored locally.

A major advantage of `dea-tools` is it comes bundled with all essential dependencies for scientific geospatial analysis, including JupyterLab, Jupyter notebooks, xarray, geopandas, NumPy, dask, and other related ecosystem libraries. When deployed within a virtual environment, `dea-tools` therefore provides a **fully managed and ready-to-use geospatial analysis environment**. This design lowers the barrier to entry for new users, enabling a self-contained environment for exploration, visualisation, and large-scale satellite data processing.

The package enables users to:

- Load, combine, and manage satellite or model datasets, including but not limited to DEA products, while handling reprojection and resolution harmonisation;
- Conduct spatial and temporal analyses directly on xarray objects (e.g., extracting temporal statistics, linear regression, and interpolation);
- Apply a large curated list of remote sensing indices such as NDVI, NDWI etc.
- Perform machine learning and segmentation workflows on geospatial datasets, with built-in training, fitting, and prediction tools that work seamlessly with xarray;
- Generate publication-quality visualisations and animations;
- Seamlessly convert between raster and vector formats using xarray–geopandas interoperation utilities.
- Apply domain-specific tools: analyse coastal change, intertidal zones, land cover, wetland and waterbody dynamics, and climate datasets.

The broader `dea-notebooks` repository, which includes `dea-tools`, provides extensive Jupyter notebook based documentation including how-to guides and longer real-world application examples. These include end-to-end workflows that demonstrate how to acquire satellite data via odc-stac, process it using `dea-tools`, and visualize outputs interactively within JupyterLab. Together, `dea-tools` and `dea-notebooks` offer a reproducible, scalable, and fully managed framework for geospatial analysis in Python, bridging the gap between foundational geospatial libraries and applied EO research.

Features

Below we highlight a non-exhaustive list of functionality within `dea-tools`. Note that each of the functions listed below has an accompanying Jupyter notebook outlining how to use the function.

Data handling

Tools for loading and manipulating both DEA satellite data, along with other providers of geospatial data, stored within dea-tools.datahandling

- `load_ard`: Load and combine multiple Geoscience Australia Landsat, Sentinel 2, or Sentinel-1 Analysis Ready Data products with `odc-stac` and optionally apply pixel quality, cloud masking and contiguity masks, and drop time steps that contain greater than a minimum proportion of good quality (e.g. non-cloudy or shadowed) pixels.
- `load_reproject`: Load and reproject all or part of a raster dataset (stored either locally or remotely) to match the crs/resolution of another dataset.

Analytical tools native to Xarray

Flexible and powerful functionality for scientific analysis with xarray objects:

- Extracting time series statistics, using the module dea-tools.temporal:
  - Generic summary statistics on any time series with `temporal_statistics`
  - Phenology with `xr_phenology`
  - Linear regression with `xr_regression`
- Validation tools within dea-tools.validation:
  - Compute common statistical metrics with `eval_metric`
  - Generate random points over an xarray object with `xr_random_sampling`

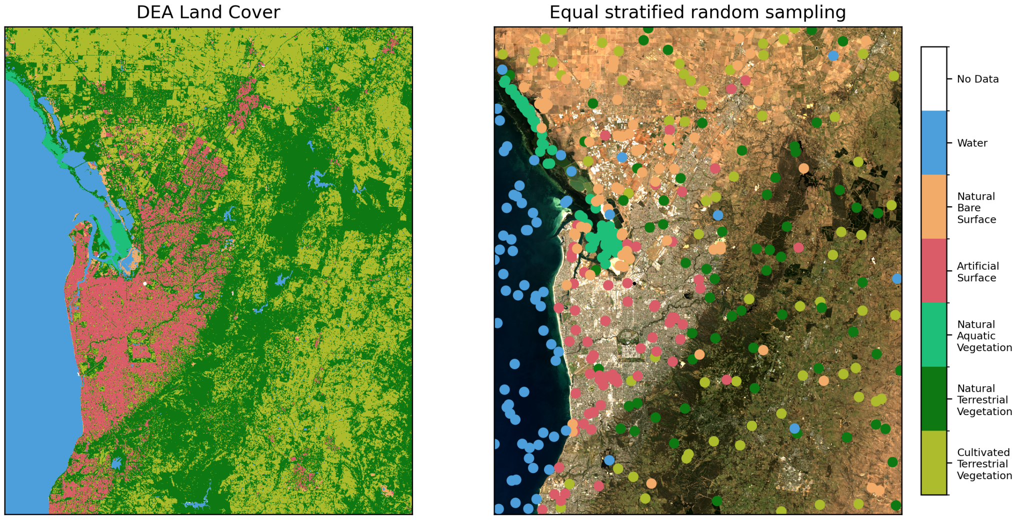

Figure 1: Example extraction of random samples from a classified xarray.DataArray using one of the sampling strategies available in the `xr_random_sampling` function.

- Spatial analysis tools within [dea-tools.spatial](https://):
  - Vectorise xarray.DataArrays into geopandas.GeoDataFrames with `xr_vectorise`
  - Rasterize geopandas.GeoDataFrames into xarray.DataArrays with `xr_rasterize`
  - Extract subpixel contours from xarray.DataArrays with `subpixel_contours`
  - Interpolate point data stored in a geopandas.GeoDataFrame into an xarray.DataArray with `xr_interpolate`

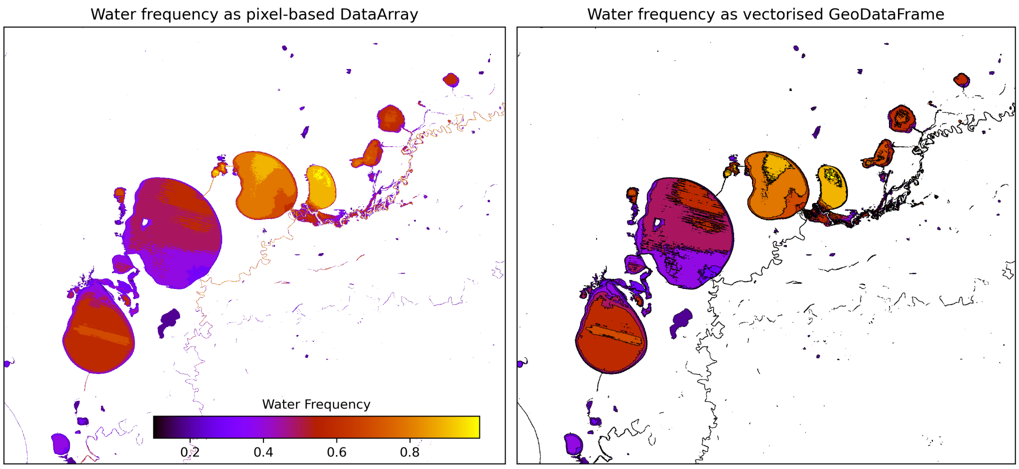

Figure 2: `dea-tools` enables easy transitions between xarray and geopandas with the `xr_rasterize` and `xr_vectorize` functions.

- Xarray wrappers for the full machine learning pipeline in dea-tools.classification:
  - prepare data for ML predictions with `sklearn_flatten` and `sklearn_unflatten`
  - Fit models with `fit_xr`
  - ML predictions with `predict_xr`
  - Handle spatial autocorrelation in cross-validation workflows with `spatial_train_test_split` and `SKCV` (spatial k-fold cross validation)

Plotting

The dea-tools.plotting library has extensive functionality for generating beautiful imagery from xarray objects.

- Highly customisable animations with `xr_animation`
- RGB plots for true and false colour composite images with `rgb`

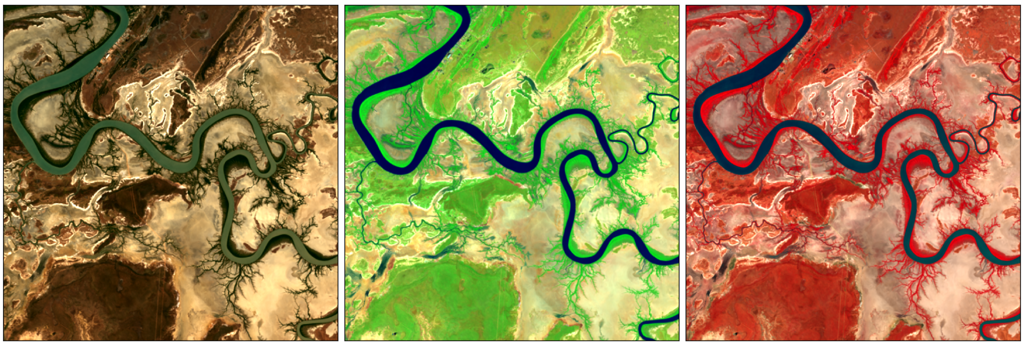

Figure 3: True and false colour composites generated with the `dea-tools.plotting.rgb` function, using DEA annual GeoMAD composite images from 2024.

Research projects

`dea-tools` and the broader `dea-notebooks` repository supports a wide range of scientific applications, from agriculture and land cover mapping to wetland, coastal, and surface water monitoring, and climate and fire analysis. The package has already underpinned more than 25 peer-reviewed studies, demonstrating its robustness and adaptability across domains. Usage of the library is recorded here.

Acknowledgements

Thanks

References